

ShopFlow AWS Project

Overall Architecture Design

18 AWS Services -- How They Connect -- ShopVerse Application

Field	Value
Version	1.0
Date	June 2026
Region	ap-south-1 (Mumbai)
Author	Manish Kumar Sharma
Application	ShopVerse -- Spring Boot 3.2.5 / Java 21
Modules	1 (IAM) through 18 (CodePipeline)
Total Services	18 AWS Services

1. Architecture Summary

The ShopFlow architecture deploys ShopVerse -- a Spring Boot 3.2.5 / Java 21 e-commerce application -- on AWS in the ap-south-1 (Mumbai) region using 18 services across 6 architectural layers. Every service is provisioned by CloudFormation and follows the principle of least privilege. No credentials are hardcoded; all secrets are auto-generated by Secrets Manager. No instance requires SSH access -- all management is via SSM Session Manager.

The deployment order follows the dependency graph: IAM and VPC first (no dependencies), then storage and secrets, then compute, then messaging and observability, then CI/CD last.

2. Architecture Layers Diagram



[Module 16] CloudWatch Logs: /shopverse/dev/app /ecs/shopflow-shopverse /aws/rds/.../slowquery
CloudWatch Alarms: ALB 5xx | ALB latency | RDS CPU | RDS storage | DLQ depth CloudWatch
Dashboard: shopflow-overview-dev (ALB + RDS + SQS widgets)

v

CI/CD LAYER

[Module 10] ECR: shopflow-shopverse (scan-on-push, lifecycle policy: keep 10 tagged images)
[Module 17] CodeBuild: Maven compile + docker build + ECR push (~5-8 min) [Module 18]
CodePipeline: GitHub push -- Source -- Build -- ECS rolling deploy (~15 min total)

v

IDENTITY AND NETWORK FOUNDATION (All Layers Depend On These)

[Module 1] IAM: 5 roles (EC2, ECS Execution, ECS Task, Lambda, CICD) | 5 policies | MFA enforced
[Module 2] VPC: 10.0.0.0/16 | 2 public subnets + 2 private subnets | IGW | NAT GW | 4 SGs

3. Complete Service Catalogue

#	Service	CloudFormation Stack	Key Resources Created	Primary Purpose for ShopVerse
1	IAM	shopflow-iam	5 roles, 5 policies, 4 groups, 11 users	Identity and access management for all services
2	VPC	shopflow-vpc	VPC, 4 subnets, IGW, route tables, 4 SGs	Network isolation -- all services run inside this VPC
3	S3	shopflow-s3-buckets	5 buckets (artifacts, cicd, rds-exports, images, spa)	Object storage for JARs, Docker artifacts, exports
4	Secrets Manager	shopflow-secrets	5 secrets with auto-generated passwords	Encrypted storage for DB, JWT, Redis credentials
5	SSM Param Store	shopflow-ssm	~20 parameters under /shopflow/dev/*	Non-sensitive config: endpoints, feature flags, TTLs
6	RDS MySQL	shopflow-rds	Subnet group + MySQL 8.0 db.t3.micro instance	Primary relational DB: orders, products, customers
7	ElastiCache Redis	shopflow-redis	Redis 7.1 cache.t3.micro replication group	Session store, product cache, rate limiter, cart
8	EC2	shopflow-ec2	LaunchTemplate + EC2 instance (t3.small, AL2023)	Spring Boot JAR host (alternative to ECS)
9	ALB	shopflow-alb	ALB + Target Group + HTTP Listener	Public HTTP entry point; health-checks Spring Boot
10	ECR	shopflow-ecr	ECR private repository + lifecycle policy	Store ShopVerse Docker images for ECS pulls
11	ECS Fargate	shopflow-ecs	ECS Cluster + Task Definition + ECS Service	Run ShopVerse as container; rolling deploy + rollback
12	SQS	shopflow-sqs	3 standard queues + 3 DLQs + 1 FIFO pair	Async queue for emails, order events, payments
13	SNS	shopflow-sns	2 topics: ops-alerts + order-fanout	Fan-out events to multiple SQS; alert ops email
14	Lambda	shopflow-lambda	2 functions: email-notifier + order-processor	Process SQS events: send emails, update inventory
15	DynamoDB	shopflow-dynamodb	2 tables: sessions (GSI) + idempotency (TTL)	JWT session store and dedup for order placement
16	CloudWatch	shopflow-monitoring	6 log groups + 5 alarms + 1 dashboard	Centralised logs, metric alarms, observability
17	CodeBuild	shopflow-codebuild	CodeBuild project (Maven + Docker, buildspec.yml)	Compile ShopVerse, build Docker image, push to ECR
18	CodePipeline	shopflow-codepipeline	Pipeline with 3 stages: Source/Build/Deploy	Automated CD: git push to main -- live in 15 min

4. Key Data Flows

4.1 User Login Flow

Step	Component	Action
1	User	POST /api/auth/login { email, password }
2	ALB	Receives on port 80, forwards to EC2/ECS on port 8080
3	Spring Boot AuthController	Validates credentials against RDS (users table)
4	JwtTokenProvider	Generates HS512 JWT signed with JWT_SECRET from Secrets Manager
5	RedisSessionStore	Stores token hash in ElastiCache Redis -- key session:{tokenHash}, TTL 24h
6	DynamoDB	Writes session item with tokenHash PK and expiresAt for fallback/GSI lookup
7	Response	JWT returned to user -- stored client-side (localStorage/cookie)

4.2 Order Placement Flow

Step	Component	Action
1	User	POST /api/orders with JWT header + Idempotency-Key header
2	ALB	Forwards to Spring Boot on port 8080
3	JwtAuthenticationFilter	Validates JWT against Redis session store
4	IdempotencyService	Checks DynamoDB idempotency table -- if key exists, return cached response
5	OrderController	Calls OrderCommandHandler via CQRS CommandBus
6	DistributedLockService	Acquires Redisson distributed lock on product IDs (prevents oversell)
7	RDS	Writes order + order_items rows; updates inventory
8	Redisson lock released	Lock released after DB commit
9	OrderKafkaProducer	Publishes order event (locally uses Kafka; AWS uses SNS instead)
10	SNS shopflow-order-fanout	Fan-out delivers to shopflow-notifications SQS + shopflow-order-events SQS
11	Lambda email-notifier	Polls notifications SQS, sends order confirmation email via SendGrid
12	Lambda order-processor	Polls order-events SQS, updates inventory counters, triggers fulfillment
13	DynamoDB	Marks idempotency key as COMPLETED with cached response

4.3 Product Search Flow

Step	Component	Action
1	User	GET /api/products/123 (no auth required -- public endpoint)
2	ALB	Forwards to Spring Boot
3	CachedProductService	@Cacheable('products') checks Redis for key products::123
4	Cache HIT	Returns cached JSON -- 30-minute TTL -- RDS not queried
5	Cache MISS	Fetches from RDS, stores in Redis with 30-minute TTL
6	GET /api/search	Elasticsearch query via ProductSearchService (keyword search)
7	POST /api/products	Admin only (Role: ADMIN required) -- invalidates Redis cache via @CacheEvict

4.4 CI/CD Deployment Flow

Step	Component	Action
1	Developer	git push origin main
2	GitHub	CodeStar Connection detects push, triggers CodePipeline
3	CodePipeline Source	Downloads source ZIP to S3 cicd-artifacts bucket
4	CodeBuild	./mvnw clean package -DskipTests -- compiles ShopVerse (5-8 min)
5	CodeBuild	docker build -t \$ECR_REPO_URI:{git-sha} . -- builds image
6	ECR	docker push -- stores versioned image, scan-on-push runs CVE scan
7	CodeBuild	Writes imagedefinitions.json: [{name:shopverse, imageUri:...}]
8	CodePipeline Deploy	ECS rolling update: new task starts with new image
9	ALB health check	/actuator/health -- must return 200 before old task stops
10	ECS Circuit Breaker	If health check fails 3x, auto-rollback to previous task definition
11	Total time	~10-15 minutes from git push to live on production

5. CloudFormation Deployment Order

Deploy stacks in this sequence -- each stack depends on the exports of stacks above it:

Order	Stack Name	Module	Exports Used From
1	shopflow-iam	IAM	(none -- first stack to deploy)
2	shopflow-vpc	VPC	(none -- first network stack)
3	shopflow-s3-buckets	S3	(none -- standalone)
4	shopflow-secrets	Secrets Manager	(none -- standalone)
5	shopflow-ssm	SSM	(none -- standalone)
6	shopflow-rds	RDS	shopflow-vpc (subnet IDs, RDS SG), shopflow-secrets (DB password ARN)
7	shopflow-redis	ElastiCache	shopflow-vpc (subnet IDs, EC2 SG), shopflow-secrets (Redis AUTH ARN)
8	shopflow-ec2	EC2	shopflow-iam (instance profile), shopflow-vpc, shopflow-s3, shopflow-ssm, shopflow-secrets
9	shopflow-alb	ALB	shopflow-vpc (public subnets, ALB SG), shopflow-ec2 (instance ID)
10	shopflow-ecr	ECR	(none -- standalone repository)
11	shopflow-ecs	ECS Fargate	shopflow-iam (task/execution roles), shopflow-vpc, shopflow-ecr, shopflow-ssm, shopflow-secrets, shopflow-alb (TG ARN)
12	shopflow-sqs	SQS	(none -- standalone)
13	shopflow-sns	SNS	shopflow-sqs (queue ARNs for subscriptions)
14	shopflow-lambda	Lambda	shopflow-iam (Lambda role), shopflow-vpc, shopflow-sqs (trigger ARNs), shopflow-s3 (code bucket)
15	shopflow-dynamodb	DynamoDB	(none -- standalone)
16	shopflow-monitoring	CloudWatch	shopflow-alb, shopflow-rds, shopflow-sqs, shopflow-sns (ops topic ARN)
17	shopflow-codebuild	CodeBuild	shopflow-iam (CICD role), shopflow-ecr (repo URI), shopflow-s3 (cicd bucket)
18	shopflow-codepipeline	CodePipeline	shopflow-codebuild (project name), shopflow-ecs (cluster + service), shopflow-s3 (cicd bucket)

6. Security Design Summary

Security Principle	Implementation
Zero hardcoded secrets	All passwords auto-generated by Secrets Manager; JWT secret is 64-char random; Redis AUTH is 32-char alphanumeric

Security Principle	Implementation
Least privilege IAM	Each service has its own role with only the permissions it needs; no wildcard actions in production policies
Network isolation	EC2, ECS, RDS, Redis, Lambda all in private subnets; only ALB in public subnets; no direct internet access
No SSH keys	EC2 managed via SSM Session Manager; ECS has no SSH surface; all access is API-based with IAM auth
Encryption everywhere	S3 (SSE-S3), RDS (AES-256 at rest), Redis (TLS in-transit + AES at rest), ECR (AES-256), Secrets Manager (KMS)
MFA enforced	IAM password policy requires MFA for all console users; no MFA = no sensitive operation allowed
Stateless auth	Spring Boot uses STATELESS session policy; JWT validated on every request against Redis session store
Idempotency	DynamoDB idempotency table prevents double order creation even under network retries
Rate limiting	Redis Lua script enforces rate limits at the application level: 10 auth/min, 200 API requests/min per user

7. Monthly Cost Estimate (dev environment)

Service	Resource	Est. Monthly Cost
RDS	db.t3.micro Single-AZ MySQL 8.0 + 20GB gp3	\$14.71
ElastiCache	cache.t3.micro Redis 7.1 single node	\$11.62
EC2	t3.small (if running 24/7)	\$15.18
ECS Fargate	0.5 vCPU / 1 GB, 1 task 24/7	\$14.45
ALB	1 ALB + ~1M LCUs/month	\$16.00
SQS	First 1M requests free; low traffic	\$0.00
SNS	First 1M requests free	\$0.00
Lambda	First 1M invocations free; low traffic	\$0.00
DynamoDB	On-demand; very low traffic	\$0.00
Secrets Manager	5 secrets x \$0.40/month	\$2.00
S3	5 buckets, ~5GB storage	\$0.15
CloudWatch	Log ingestion ~1GB, custom metrics	\$5.00
ECR	First 500MB free; ~1GB total	\$0.10
CodeBuild	Free tier 100 min/month; ~2-3 builds/day	\$5.00

Service	Resource	Est. Monthly Cost
NAT Gateway	Required for private subnets to reach internet	\$32.00
TOTAL		~\$116 / month (dev)

Note: Production environment will cost significantly more due to Multi-AZ RDS, multiple ECS tasks, higher traffic, and production-grade data transfer. NAT Gateway is the dominant cost in dev -- consider a NAT Instance (~\$4/month) for learning environments to reduce this cost.